ASSIGNMENT-01

CAMPUS SMART-PARKING SYSTEM

Design, Implement and Validate an Object-Oriented Model

Assignment Report – Tasks 1 to 12 (Total Marks: 50)

Item	Details
Student name	**Jothika M**
Register number	**192324235**
Course	**Programming in Java (CSA0917)**
Java version used	**OpenJDK 21**
GitHub repository	**CSA0917**
GitHub link	**https://github.com/Jothika-M25/CSA0917**

Task 1 – Understand the Problem and Identify the Objects

Bloom's level: K4 – Analyse | Marks: 4

1.1 Inputs and outputs

The Campus Smart-Parking System is a transaction-processing program. Every operation either admits a vehicle or releases one, and each leaves the facility in a consistent state.

Operation	Inputs received	Outputs produced
Entry	Registration number, vehicle type, user type (student / staff / visitor), pass status, entry time	Allocated slot (floor, slot), unique ticket number, entry timestamp
Exit	Registration number (or ticket number), exit time, kWh used (EVs only)	Released slot, duration in minutes, parking fee, charging fee, receipt
Query	Optional floor number	Free slots per floor, occupancy, day's collection

1.2 Nouns become classes, verbs become methods

Noun (candidate class)	Why it is needed (responsibility)	Verbs (methods)
Vehicle (abstract) → TwoWheeler, Car, ElectricCar	Holds the registration number and knows its own hourly rate. Different types price differently, so type-specific behaviour lives here.	calculateFee(), ratePerHour(), getRegNo()
ParkingSlot / slots array	Represents one numbered position that can hold at most one vehicle; enforces the "no two vehicles in a slot" rule.	isFree(), occupy(), vacate()
ParkingTicket	Proof of entry: unique number, vehicle, slot, entry time. Needed at exit to compute duration.	getTicketNo(), getEntryMinute()
ParkingFacility / ParkingLot	Owns floors, slots and active tickets; coordinates entry and exit. It is the only class that changes slot state.	enter(), exit(), allocate(), release(), freeSlots()

Noun (candidate class)	Why it is needed (responsibility)	Verbs (methods)
User / UserType	Decides whether a pass applies; staff and students with valid pass park free, visitors pay.	hasFreePass()
Chargeable (interface)	Capability only some vehicles have (EV charging). Keeps charging out of Vehicle.	chargingFee()

A candidate class is retained only if it carries data or rules that cannot be placed elsewhere without duplication. “Floor” was rejected as a separate class because a floor has no behaviour of its own beyond being a row of the slot array; “Fee” was rejected because the fee rule depends on the vehicle type and therefore belongs inside the Vehicle hierarchy.

1.3 Business rules

- A slot cannot hold two vehicles at the same time.
- A ticket number is never reused; numbers only increase.
- A registration number can have only one active ticket (duplicate entry is rejected).
- A vehicle can only exit if it has an active ticket; a slot can only be released once.
- Fee = started hours × hourly rate; any started hour is billed in full; a zero-minute stay is free; the fee for one calendar day is capped at 8 × hourly rate.
- Students and staff holding a valid monthly pass pay nothing; visitors always pay the full fee.
- A negative duration or an invalid floor/slot index is rejected with an exception.

1.4 Assumptions

- Time is stored as minutes since midnight (entry and exit on the same or consecutive days).
- Slots are allocated first-free-first (lowest floor, lowest slot number).
- An EV may charge in its parking slot; charging is billed at Rs 12 per kWh in addition to parking.
- Rates: two-wheeler Rs 10/h, car Rs 30/h, EV Rs 30/h + charging.

1.5 Alternative concept and complexity analysis – parallel arrays of Strings

If vehicles were plain Strings in parallel arrays, the program would hold String[] regNo, String[] type and int[] entryTime with a shared index. Each new attribute means a new array that must be kept in step manually.

Parallel-array sketch

```
String[] regNo = new String[N];   String[] type = new String[N];   int[] entryTime = new int[N];
int find(String r) {
    for (int i = 0; i < count; i++) if (regNo[i].equals(r)) return i;
    return -1;
}
```

Criterion	Object design (Vehicle objects)	Parallel arrays
Find by registration number	O(n) over an array of references; O(1) average if indexed by a HashMap	O(n) linear scan; no way to index without extra structure
Add a new attribute	Add one field to Vehicle; no other class changes	Add a new array and edit every routine that moves or deletes entries – easy to desynchronise
Space per vehicle	Object header (~12–16 B) + references; fields stay together (better cohesion)	No object header, but 3+ separate arrays; a few bytes less per vehicle
Type safety / behaviour	Fee rule is a method on the object	Behaviour needs a switch on the type String; typos compile silently

Conclusion: both designs search in O(n); the object design costs a little more memory per vehicle but wins decisively on maintainability and on adding behaviour.

Task 2 – Choose the Data Representation

Bloom's level: K4 – Analyse | Marks: 3

2.1 Data types chosen

Field	Java type	Reason
Slot number, floor index	int	Small non-negative whole numbers (≤ 100 per floor); used directly as array indices.
Vehicle number	String	Alphanumeric (e.g. TN01AB1111); normalised to upper case to make comparison reliable.

Field	Java type	Reason
Entry time	int (minutes since midnight) or LocalTime	Integer arithmetic gives exact durations; LocalTime used where clock rules matter (Task 12).
Duration	int (minutes)	Whole minutes only; avoids floating-point error.
Fee	double	Rs amounts may become fractional if rates change; BigDecimal would be safer for real money (noted as a limitation).
Ticket number	int (static counter)	Counts well past facility capacity; long if issuing beyond 2 billion.
Has pass	boolean	Two states only.

2.2 Floors and slots: jagged array

The facility is expressed as Vehicle[][] slots (or String[][] in the supplied exercise). A jagged array is chosen because Task 12 requires floors with different slot counts; a rectangular 2-D array would waste memory and need a separate "valid slots" array. Row f is created with its own length: slots[f] = new String[slotsPerFloor[f]].

2.3 Input handling

Command-line arguments suit fixed start-up values (floors and slots per floor); a Scanner suits interactive entry and exit transactions. Both are acceptable; the test harness in this report uses hard-coded inputs so that results are reproducible.

2.4 Casting and precision – an example that gives a wrong fee

Billing converts minutes to hours. A car parked for 130 minutes must pay for 3 started hours (Rs 90). Integer division gives 130 / 60 = 2, so the fee silently becomes Rs 60. Casting the result after the division does not repair it, because the fraction is already lost.

CastDemo.java

```
public class CastDemo {
    public static void main(String[] args) {
        int minutes = 130; double rate = 30;
          double wrong1 = (minutes / 60) * rate;
        double wrong2 = (double) (minutes / 60) * rate;
            double wrong3 = Math.ceil(minutes / 60) * rate;
```

```
            double right1 = ((minutes + 59) / 60) * rate;
            double right2 = Math.ceil(minutes / 60.0) * rate;
        System.out.println("Wrong (int division)        : Rs " + wrong1);
        System.out.println("Wrong (cast after division) : Rs " + wrong2);
        System.out.println("Wrong (ceil of int division): Rs " + wrong3);
        System.out.println("Right (integer ceiling)     : Rs " + right1);
        System.out.println("Right (ceil of 130/60.0)    : Rs " + right2);
        long big = 3_000_000_000L; int narrowed = (int) big;
        System.out.println("long -> int narrowing: " + big + " -> " + narrowed);
        Integer a = 127, b = 127, c = 1000, d = 1000;
        System.out.println("127==127 : " + (a == b) + " | 1000==1000 : " + (c == d)
                + " | 1000.equals(1000) : " + c.equals(d));
    }
}
```

Output

```
Wrong (int division)        : Rs 60.0
Wrong (cast after division) : Rs 60.0
Wrong (ceil of int division): Rs 60.0
Right (integer ceiling)     : Rs 90.0
Right (ceil of 130/60.0)    : Rs 90.0
long -> int narrowing: 3000000000 -> -1294967296
127==127 : true | 1000==1000 : false | 1000.equals(1000) : true
```

Resolution: the design uses the integer ceiling idiom (minutes + 59) / 60, which needs no floating point at all. The alternative Math.ceil(minutes / 60.0) is also correct because 60.0 forces real division. The same demo shows two other risks: narrowing long → int overflows to a negative number, and Integer objects above 127 compare false with ==.

2.5 Alternative concept – 1-D array or ArrayList<String[]>

With one flat array, slot (f, s) maps to index = f × slotsPerFloor + s. This only works when every floor has the same size. For floors of different sizes the code needs a prefix-sum array start[f] so that index = start[f] + s, and the floor of a given index requires a search. An ArrayList<String[]> keeps the jagged shape and can grow, but adds an indirection per row.

Representation	Locate slot (f, s)	Space	Different floor sizes
Jagged String[][]	O(1): slots[f][s]	Σ sizes + F row references	Natural
1-D array, f × S + s	O(1) arithmetic	F × S (wastes unused slots if sizes differ)	Breaks – needs prefix sums
ArrayList<String[]>	O(1) get(f) then [s]	Σ sizes + list overhead	Works; resizable

Task 3 – Design the Control Logic

Bloom's level: K4 – Analyse | Marks: 4

3.1 Searching for a free slot

The program scans floors from 0 upwards and slots from 0 upwards. It stops at the first null entry (return inside the nested loop), so on average much less than the whole facility is examined. If both loops finish, the facility is full and null is returned. Using return instead of break avoids needing a found-flag and a second exit test for the outer loop.

3.2 Fee slabs

Fee slabs are expressed with if–else on the minute count: zero minutes is free; otherwise started hours × rate, limited to a daily maximum of 8 × rate; whole days beyond 24 hours each cost the daily maximum. A switch is used only on the user type or vehicle category, never for numeric ranges. continue skips occupied slots in the search for special slot types; return ends early in release() as soon as validation fails.

3.3 Pseudocode

Pseudocode 1 – slot allocation

```
ALGORITHM AllocateSlot
INPUT  : vehicleNo (String), slots[0..F-1][0..S(f)-1]
OUTPUT : {floor, slot} or NULL
BEGIN
  FOR floor <- 0 TO F-1 DO
    FOR slot <- 0 TO length(slots[floor])-1 DO
      IF slots[floor][slot] = NULL THEN
        slots[floor][slot] <- vehicleNo
        RETURN {floor, slot}
      END IF
    END FOR
```

```
    END FOR
    RETURN NULL                    // facility is full
END
```

Pseudocode 2 – fee computation

```
ALGORITHM ComputeFee
INPUT  : minutes (INTEGER >= 0), rate (hourly rate), hasFreePass (BOOLEAN)
OUTPUT : fee (REAL)
BEGIN
    IF hasFreePass THEN RETURN 0
    IF minutes < 0 THEN RAISE InvalidDuration
    IF minutes = 0 THEN RETURN 0
    dailyMax  <- 8 * rate
    days      <- minutes DIV 1440
    rest      <- minutes MOD 1440
    hours     <- (rest + 59) DIV 60
    restFee   <- MIN(hours * rate, dailyMax)
    RETURN days * dailyMax + restFee
END
```

The pseudocode follows a consistent algorithmic style: named ALGORITHM, declared INPUT/OUTPUT, BEGIN/END blocks, upper-case keywords and <- for assignment. If the SIMATS standard document specifies a different keyword set, only the keywords need renaming; the logic is unaffected.

3.4 Complete the Code – ParkingFacility

TODO 1 and TODO 2 were replaced; main() is unchanged. The program was compiled and executed (Java 21).

ParkingFacility.java

```
public class ParkingFacility {
      private final String[][] slots;
    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            slots[f] = new String[slotsPerFloor[f]];
```

```
        }
    }
     public int[] allocate(String vehicleNo) {
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
            }
        }
        return null;
    }
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
                + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

Output (matches the expected output exactly)

```
[0, 0]
```

```
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

When no slot is free, allocate() returns null and the caller reports “Parking full”; in the full system (Task 11) this becomes an IllegalStateException so that a failure cannot be ignored.

3.5 Alternative concept – free-slot counts and a stack of free slots

Keep int[] freeCount (one per floor) and a stack of free slot numbers per floor. allocate() picks the first floor with freeCount > 0 and pops a slot number; release() pushes the slot number back and increments the count. Allocation is then no longer “lowest slot first”, unless a priority queue (O(log S)) is used.

Operation	Scan approach (implemented)	Counter + stack approach
allocate()	O(F × S) worst case, O(1) best case	O(F) to find a floor with space (O(1) with a global stack)
release()	O(1)	O(1) push
Extra space	O(1)	O(F) counters + O(F × S) stack entries

Task 4 – Design Classes, Objects and Constructors

Bloom’s level: K4 – Analyse | Marks: 4

4.1 Fields and methods

Class	Fields	Key methods
Vehicle (abstract)	private final String regNo	getRegNo(), calculateFee(int), abstract ratePerHour(), equals(), hashCode(), toString()
ParkingTicket	ticketNo, vehicle, user, validPass, floor, slot, entryMinute; static counter	getters; constructor assigns the next ticket number

Class	Fields	Key methods
ParkingLot	Vehicle[][] slots, Map<String,ParkingTicket> active, double collection	enter(), exit(), freeSlots(), getCollection()
Slot (demo class)	floor, number, occupied	setters with validation, toString()

4.2 Constructors, this and chaining

The demo class Slot has four constructors. The default constructor delegates with this(0, 0); the two-argument one delegates to the three-argument one with this(floor, number, false); only the three-argument constructor does real work (calling the validating setters). This keeps validation in exactly one place, so a new rule never has to be copied into several constructors. The copy constructor Slot(Slot other) builds a new, independent object from another one.

MiscDemo.java (Tasks 4, 5, 8 and 10 demonstrations)

```
import java.time.LocalTime;
import java.time.Duration;

class Slot {
    private int floor, number; private boolean occupied;
    Slot() { this(0, 0); }
    Slot(int floor, int number) { this(floor, number, false); }
    Slot(int floor, int number, boolean occupied) { setFloor(floor); setNumber(number); this.occupied = occupied; }
    Slot(Slot other) { this(other.floor, other.number, other.occupied); }
    void setFloor(int f) { if (f < 0) throw new IllegalArgumentException("Invalid floor: " + f); this.floor = f; }
    void setNumber(int n) { if (n < 0) throw new IllegalArgumentException("Invalid slot: " + n); this.number = n; }
    @Override public String toString() { return "Slot[F" + floor + "-S" + number + (occupied ? ", occupied]" : ", free]"); }
}
class Ticket {
    private static int counter = 0;
    static int issued() { return counter; }
    private final int no = ++counter;
```

```
    int getNo() { return no; }
}

class Fee {
    static double calculateFee(int minutes) { return ((minutes + 59) / 60) * 30.0; }
    static double calculateFee(LocalTime in, LocalTime out) {
        return calculateFee((int) Duration.between(in, out).toMinutes());
    }
    static double calculateFee(double hours) { return Math.ceil(hours) * 30.0; }
}
abstract class V { abstract String kind(); }
class Bike extends V { String kind() { return "Bike.kind() executed"; } }
class Lorry extends V { String kind() { return "Lorry.kind() executed"; } }
public class MiscDemo {
    public static void main(String[] args) {
        System.out.println("-- Task 4: constructors --");
        Slot s1 = new Slot(), s2 = new Slot(2, 7), s3 = new Slot(s2);
        System.out.println(s1 + " " + s2 + " " + s3 + "  same object? " + (s2 == s3));
        try { new Slot(-1, 3); } catch (IllegalArgumentException e) { System.out.println("Rejected: " +
e.getMessage()); }
        System.out.println("-- Task 5: static counter shared by two facilities --");
        Ticket n1 = new Ticket(), n2 = new Ticket(), s1t = new Ticket();
        System.out.println("North tickets 1,2 and South ticket -> " + n1.getNo() + "," + n2.getNo() + ","
+ s1t.getNo()
                + " | total issued = " + Ticket.issued());
        Ticket gone = new Ticket(); gone = null;       // now unreachable -> eligible for GC
        System.out.println("Reference set to null; object is now eligible for garbage collection");
        System.out.println("-- Task 8: overload resolution --");
        System.out.println("calculateFee(130)           = " + Fee.calculateFee(130));
        System.out.println("calculateFee(10:00,  12:10)     =  "  +  Fee.calculateFee(LocalTime.of(10,  0),
LocalTime.of(12, 10)));
        System.out.println("calculateFee(2.2)           = " + Fee.calculateFee(2.2));
        System.out.println("-- Task 8: dynamic dispatch trace --");
        V[] arr = { new Bike(), new Lorry() };
        for (V v : arr)
            System.out.println("declared=V, actual=" + v.getClass().getSimpleName() + " -> " + v.kind());
```

```
        System.out.println("-- Task 10: wrapper-class risk --");
        Integer a = 128, b = 128;
        System.out.println("a == b : " + (a == b) + "   a.equals(b) : " + a.equals(b));
        Integer maybeNull = null;
        try { int x = maybeNull; } catch (NullPointerException e) { System.out.println("Unboxing null ->
NullPointerException"); }
    }
}
```

Output

```
-- Task 4: constructors --
Slot[F0-S0, free] Slot[F2-S7, free] Slot[F2-S7, free]  same object? false
Rejected: Invalid floor: -1
-- Task 5: static counter shared by two facilities --
North tickets 1,2 and South ticket -> 1,2,3 | total issued = 3
Reference set to null; object is now eligible for garbage collection
-- Task 8: overload resolution --
calculateFee(130)           = 90.0
calculateFee(10:00, 12:10)   = 90.0
calculateFee(2.2)           = 90.0
-- Task 8: dynamic dispatch trace --
declared=V, actual=Bike -> Bike.kind() executed
declared=V, actual=Lorry -> Lorry.kind() executed
-- Task 10: wrapper-class risk --
a == b : false   a.equals(b) : true
Unboxing null -> NullPointerException
```

4.3 Copy constructor and memory

A copy constructor is useful when re-issuing a duplicate ticket: the duplicate has the same data but is a separate object, so changing one does not change the other (the output shows s2 == s3 is false). Where objects are immutable (ParkingTicket fields are final), sharing a reference is safer and cheaper than copying.

A reference variable such as Slot s2 lives on the stack of the running method; the Slot object itself lives on the heap. Assigning s3 = s2 copies the reference, not the object; new Slot(s2) creates a second heap object. A static field such as the ticket counter lives in the class's static storage, shared by all instances.

4.4 Alternative concept – default constructor + setters, or static factories

Approach	How it guarantees a valid object	Cost of creating n tickets
Overloaded constructors (used)	Object is valid the moment it exists; fields can be final	Time O(n); space O(n) objects
Default constructor + setters	Object exists in an invalid half-built state until every setter is called; cannot use final fields; callers can forget a setter	Time O(n) but with extra method calls; space O(n)
Static factory methods (e.g. Ticket.forVisitor(...))	Named, can validate and return cached or subclass instances; constructor can be private	Time O(n); space O(n) (O(1) extra per call if instances are cached)

Task 5 – Apply static Members and the Object Life Cycle

Bloom's level: K4 – Analyse | Marks: 3

5.1 Where static is required

The ticket counter must be static because it belongs to the class as a whole: every ParkingTicket created, by any facility, must receive a number larger than all previous ones. If it were an instance variable, each new ticket would start again at 1, and ticket numbers would be reused – breaking a business rule. A static total-occupancy count can likewise summarise all facilities. In the Task 5 demo (MiscDemo above) the north gate issues tickets 1 and 2, the south gate receives 3, and Ticket.issued() reports 3.

Static methods fit behaviour that needs no object state, such as Ticket.issued() or a pure helper that converts minutes to started hours. A static initialiser block is appropriate for one-time set-up, for example loading the rate table when the class is first loaded.

5.2 Object life cycle

A ParkingTicket is created at entry and stored in the active-ticket map. When the vehicle exits, the map entry is removed; if no other reference remains (the receipt keeps only copied values), the ticket becomes unreachable and eligible for garbage collection. Setting a reference to null, as the demo does with gone = null, has the same effect. The JVM decides when memory is actually reclaimed, so the program never depends on finalisation.

5.3 Passing objects

Java passes object references by value. exit(String regNo, …) receives a copy of the reference to a String; ParkingLot.enter() returns a reference to the new ticket. A method that modifies an object's state

through its reference changes the caller's object, but reassigning the parameter does not. Tickets are immutable, so a callee cannot corrupt them.

5.4 Alternative concept – counter as an instance variable

If the counter were an instance variable of ParkingFacility, two facilities (north and south gates) would each generate ticket 1001, 1002, … and duplicates would appear across the campus. Solution (a): keep it static (one shared counter, thread-safe with AtomicInteger). Solution (b): keep it per facility but prefix the ticket with a facility code (N-1001, S-1001), which is also unique.

Design	Space for k facilities and n tickets	Uniqueness
Static counter	O(1) for the counter + O(n) tickets	Campus-wide
Instance counter per facility	O(k) counters + O(n) tickets	Only within one facility unless a prefix is added

Task 6 – Design the Inheritance Hierarchy

Bloom's level: K4 – Analyse | Marks: 4

6.1 Hierarchy and type of inheritance

Vehicle is the abstract superclass. TwoWheeler and Car extend it (hierarchical inheritance); ElectricCar extends Car (multilevel inheritance). ElectricCar also implements the Chargeable interface. Java does not allow multiple class inheritance, so the extra capability is expressed through an interface. A user hierarchy (Student, Staff, Visitor) was considered but replaced by the UserType enum, because the three user kinds differ only in data (pass eligibility), not in behaviour.

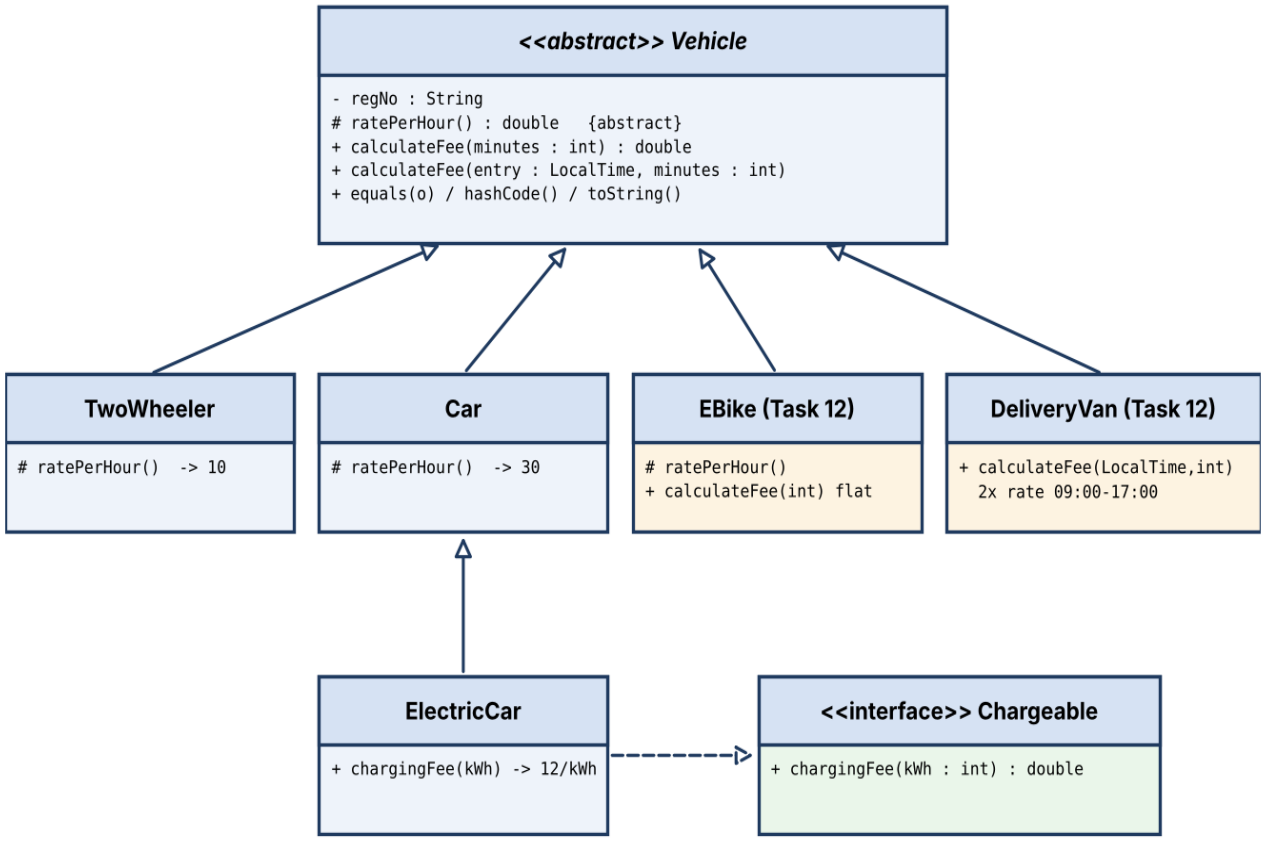


Figure 1 – UML class diagram of the vehicle hierarchy (orange classes are added in Task 12).

6.2 What is inherited and what is added

Class	Inherited	Added / specialised
TwoWheeler	regNo, getRegNo(), calculateFee(int), equals(), hashCode()	ratePerHour() = 10
Car	same as above	ratePerHour() = 30
ElectricCar	everything from Car (including rate 30)	chargingFee(kWh) = kWh × 12 (Chargeable)

super is used in every constructor (super(regNo)) because the private field regNo can only be set by the Vehicle constructor. super.method() would be used inside an override to reuse the parent's result, for example an EV class that calls super.calculateFee(minutes) and adds a surcharge.

6.3 Constructor execution order – trace

When new ElectricCarT("TN03C3", 40) runs, the constructor of ElectricCarT is entered first, but its first statement super(r) hands control up the chain before any of its own body executes. The superclass constructors therefore finish first, from the top down.

CtorTrace.java

```
class VehicleT {
    VehicleT(String r) { System.out.println("1. Vehicle constructor  (regNo=" + r + ")"); }
}
class CarT extends VehicleT {
    CarT(String r) { super(r); System.out.println("2. Car constructor"); }
}
class ElectricCarT extends CarT {
    ElectricCarT(String r, int kWh) { super(r); System.out.println("3. ElectricCar constructor (battery="
+ kWh + " kWh)"); }
}
public class CtorTrace {
    public static void main(String[] args) {
        System.out.println("Creating new ElectricCarT(\"TN03C3\", 40):");
        new ElectricCarT("TN03C3", 40);
    }
}
```

Output

```
Creating new ElectricCarT("TN03C3", 40):
1. Vehicle constructor  (regNo=TN03C3)
2. Car constructor
3. ElectricCar constructor (battery=40 kWh)
```

6.4 Alternative concept – composition (Vehicle has-a VehicleType)

A single concrete Vehicle class would hold a VehicleType object (name, hourly rate). Adding a new type means creating one new VehicleType instance (or a table row) instead of a new subclass. Fee calculation is vehicle.getType().getRate() × hours.

Criterion	Inheritance (used)	Composition

Criterion	Inheritance (used)	Composition
Add a new vehicle type	New subclass file; no existing class edited	New VehicleType data; no new class at all
Fee computation time	O(1) virtual call	O(1) field read + multiply
Space per vehicle	regNo reference + object header	regNo + one extra reference to a shared VehicleType (shared, so no per-vehicle copy)
Type-specific behaviour (EV charging, van peak rule)	Natural – override a method	Awkward – needs strategy objects or flags

Conclusion: composition is better when types differ only by data; inheritance is better when they differ by behaviour. Task 12 shows the behaviour difference (delivery van time rule), which is why inheritance plus a pricing hook was kept.

Task 7 – Override and Specialise Behaviour

Bloom's level: K4 – Analyse | Marks: 4

7.1 What is overridden and why

ratePerHour() is abstract in Vehicle, so every concrete subclass must override it. This is the one point where vehicle types differ in the base fee rule. calculateFee() is defined once in Vehicle and reused; subclasses override it only when the pricing rule changes shape (EBike flat fee and DeliveryVan time rule in Task 12). toString(), equals() and hashCode() are overridden from Object.

7.2 Rules of overriding

- **Signature:** the name and parameter list must be identical; otherwise the method is an overload, not an override.
- **Return type:** must be the same or a covariant (more specific) type.
- **Access:** may be widened (protected → public) but not narrowed.
- **@Override:** makes the compiler report an error if the signature does not really match – it catches typos such as ratePerhour().
- **Exceptions:** an override cannot declare broader checked exceptions.

7.3 Use of final

- **final field:** regNo is final – a vehicle's registration never changes after construction.
- **final method:** dailyMax() (in the full system) is final so no subclass can change the daily-cap rule by accident.
- **final class:** a value class such as a Money type would be final so it cannot be subclassed and made mutable.

7.4 equals(), hashCode() and toString()

Two Vehicle objects are equal if their registration numbers are equal, regardless of subclass. hashCode() uses the same field, so equal objects have equal hash codes – the contract required by HashMap. toString() returns "Car[TN02B2]" for readable logs. The FeeDemo output in Task 8 prints true for new Car("TN02B2").equals(new ElectricCar("TN02B2")).

7.5 Alternative concept – one calculateFee() with a switch on a type field

Switch-based version

```
double calculateFee(int minutes) {
    switch (type) {                          // every new type edits THIS method
        case "TWO_WHEELER": return hours(minutes) * 10;
        case "CAR":         return hours(minutes) * 30;
        case "EV":          return hours(minutes) * 30;
        default: throw new IllegalArgumentException("Unknown type " + type);
    }
}
```

Criterion	Overriding (used)	Switch on type
Time per fee calculation, k types	O(1) virtual dispatch	O(1) with a compiled switch on strings/enums; O(k) if written as if–else chain
Classes changed when a type is added	0 existing (1 new subclass)	1 existing (Vehicle) – every time, retesting everything
Risk	Compiler forces ratePerHour() to be written	Forgotten case or misspelt String is found only at run time

Task 8 – Apply Polymorphism

Bloom's level: K4 – Analyse | Marks: 4

8.1 Compile-time polymorphism (overloading)

Fee has three overloaded calculateFee methods: (int minutes), (LocalTime in, LocalTime out) and (double hours). The compiler chooses by the number and static types of the arguments: 130 is an int, so the int version is chosen; two LocalTime arguments select the time-range version; 2.2 is a double, so the double version is chosen. An int argument never needs widening when an exact int overload exists. The output is in the MiscDemo listing above (all three give Rs 90.0).

8.2 Runtime polymorphism and upcasting

A single Vehicle[] array holds a TwoWheeler, a Car and an ElectricCar – each element is upcast implicitly to Vehicle. The loop calls v.calculateFee(130) with no type tests; the JVM selects ratePerHour() from the actual object at run time (dynamic method dispatch). Long if–else chains such as "if (v instanceof Car) … else if …" are avoided, so adding a new vehicle never requires editing the loop.

Dispatch trace for v = new ElectricCar("TN03C3") (third array element): declared type = Vehicle; actual object type = ElectricCar; v.calculateFee(130) is found in Vehicle (inherited, not overridden); inside it ratePerHour() is called, and the JVM looks first in ElectricCar (not found), then in Car – Car.ratePerHour() = 30 executes. 3 hours × 30 = Rs 90, plus charging (instanceof Chargeable) 10 × 12 = Rs 120, total Rs 210.

8.3 Complete the Code – FeeDemo

The three TODOs were completed: TODO 1 uses (minutes + 59) / 60 for the integer ceiling; TODO 2 is the Car constructor with super and the override returning 30; TODO 3 implements chargingFee() at Rs 12 per kWh.

FeeDemo.java

```
interface Chargeable {
    double chargingFee(int kWh);
}

abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();
```

```
    /** Every started hour is charged in full. */
    public double calculateFee(int minutes) {
        int hours = (minutes + 59) / 60;          // ceiling using integer arithmetic only
        return hours * ratePerHour();
    }

    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 10; }
}

class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}

class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override public double chargingFee(int kWh) { return kWh * 12; }
}

public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"),
                             new ElectricCar("TN03C3") };
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);          // dynamic method dispatch
            if (v instanceof Chargeable c) fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo()
```

```
                + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
    }
}
```

Output (matches the expected output exactly)

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Why the last line prints true: equals() compares only the registration number and uses instanceof Vehicle, so a Car and an ElectricCar with the same number are equal. This is the correct behaviour for this system: a registration plate identifies one physical vehicle, and the same plate must never appear in two active tickets at once. The risk of such a symmetric, subclass-agnostic equals() is that it must be written once in the base class and never overridden in subclasses.

8.4 Alternative concept – instanceof checks and casting

instanceof chain

```
double fee;
if (v instanceof ElectricCar)        fee = ((ElectricCar) v).calculateFee(130) + ((ElectricCar) v).chargingFee(10);
else if (v instanceof Car)        fee = ((Car) v).calculateFee(130);
else if (v instanceof TwoWheeler)  fee = ((TwoWheeler) v).calculateFee(130);
else throw new IllegalStateException();   // ElectricCar MUST be tested before Car
```

Criterion	Dynamic dispatch	instanceof + cast
Time per call, k vehicle types	O(1)	Up to k type tests: O(k)
Code size	Constant loop body	Grows with every new type
Maintenance	New subclass only	Every such chain in the program must be edited; order of tests matters (ElectricCar before Car)

Task 9 – Introduce Abstraction

Bloom's level: K4 – Analyse | Marks: 4

9.1 Abstract class versus interface

Vehicle is abstract because "a vehicle" on its own is never parked – only a concrete TwoWheeler or Car is. It carries shared state (the final regNo), a constructor and a complete method (calculateFee), while leaving one decision abstract: protected abstract double ratePerHour(). A class can extend only one abstract class, which is exactly right for the "is-a" relationship.

Chargeable is an interface because charging is a capability that cuts across the hierarchy: a future electric two-wheeler or electric van could be Chargeable without sharing a parent with ElectricCar. An interface has no state, so any class can add it. Other suitable interfaces are Payable (fee-paying users) and Pass (Task 12).

Use an abstract class when…	Use an interface when…
Subclasses share fields and constructor logic	Unrelated classes share only a capability
There is a natural "is-a" parent	A class needs several behaviours at once (multiple interfaces allowed)
You want protected helpers and partial implementation	You want a contract that callers depend on without knowing the class

A class obtains multiple behaviours through interfaces: class ElectricCar extends Car implements Chargeable, Payable. A default method is useful for adding a method to an interface without breaking existing implementers (for example, a default describe() in Pass); a static method suits a factory or utility, such as Pass.none().

9.2 Alternative concept – Vehicle as an interface, Chargeable as an abstract class

If Vehicle were an interface with default methods, it could not hold the private final regNo field (interfaces have only constants), could not have a constructor to validate it, and each implementing class would have to store its own regNo. If Chargeable were an abstract class, ElectricCar could no longer extend Car, because Java permits only one superclass – the design would break, or ElectricCar would have to give up its Car parentage.

Feature	Abstract class	Interface (with default methods)
Instance state / constructor	Yes	No (constants only)

Feature	Abstract class	Interface (with default methods)
Multiple inheritance of type	No	Yes
Time per method call	Virtual call, O(1)	Virtual call, O(1) – negligible difference after JIT
Space per object	Fields of the class	Fields must be duplicated in every implementer

Task 10 – Enforce Encapsulation through Packages and Access Modifiers

Bloom's level: K4 – Analyse | Marks: 3

10.1 Package structure

Package layout

```
CSA09-<RegisterNumber>-Assignment1/
└── src/parking/
    ├── model/     Vehicle, TwoWheeler, Car, ElectricCar, EBike, DeliveryVan,
    │              Chargeable, ParkingTicket, UserType, Pass, MonthlyPass, OneDayPass
    ├── service/   ParkingLot (allocation, release, billing), Facility
    └── app/       ParkingSystem (main – tests), Task12Demo
```

model holds data and rules, service holds the operations that use the model, and app holds entry points. Dependencies point one way only (app → service → model), so the model can be tested without the application.

10.2 Access modifiers

Member	Modifier	Why
Vehicle.regNo	private final	Only the constructor may set it; no outside code can corrupt it.
Vehicle.ratePerHour()	protected abstract	Subclasses must implement it; callers use calculateFee() instead.
Vehicle.calculateFee()	public	The behaviour other packages need.
ParkingTicket constructor	package-private	Only the service package should issue tickets.

Member	Modifier	Why
ParkingLot.slots, active	private	Slot state may change only through enter()/exit().
ParkingTicket.counter	private static	Only the constructor increments it.

10.3 Validating getters and setters

The Slot class in Task 4 rejects a negative floor or slot number: new Slot(-1, 3) throws IllegalArgumentException ("Invalid floor: -1"). Vehicle's constructor rejects blank registration numbers and calculateFee() rejects negative durations; ParkingLot.exit() rejects unknown registrations. All checks run before any state changes, so an object is never left half-updated.

10.4 Wrapper classes and autoboxing

Integer, Double and Boolean wrappers are needed for collections (Map<String, Integer>). Autoboxing converts int → Integer automatically, which hides two risks, both shown in the demo output: Integer values above 127 are separate objects, so a == b is false while a.equals(b) is true; and unboxing a null Integer throws NullPointerException. The system therefore always compares wrapper values with equals() and checks for null before unboxing.

10.5 Alternative concept – public fields, validation by callers

With public fields, every caller would have to validate before writing: if (slot >= 0 && slot < n) { … }. One forgotten check lets an invalid value in (for example vehicle.regNo = null), and the bug appears far from its cause. Validation inside the setter runs once per update in O(1); with public fields the same O(1) check is repeated at every call site (for example 12 places if 12 methods update slots), and every new rule must be added to all of them.

Criterion	Private + validating setter	Public field
Cost per update	O(1), one check	O(1) per call site, if the caller remembers
Places the check is written	1	Every caller (n places)
Can invalid state occur?	No	Yes

Task 11 – Implement, Test and Defend the Design

Bloom's level: K5 – Evaluate | Marks: 5

11.1 Complete implementation

The following program implements the whole system – the Vehicle hierarchy, ParkingTicket with a static counter, ParkingLot with a jagged array and a HashMap of active tickets – and includes a test harness that prints the expected and actual value of each case. In the GitHub repository the classes are split into the packages in Task 10; they are shown in one file here so that it can be run directly.

ParkingSystem.java

```
import java.util.*;

interface Chargeable { double chargingFee(int kWh); }

abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) {
        if (regNo == null || regNo.isBlank()) throw new IllegalArgumentException("Registration number required");
        this.regNo = regNo.trim().toUpperCase();
    }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();
    public final double dailyMax() { return ratePerHour() * 8; }

    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Duration cannot be negative");
        if (minutes == 0) return 0;                    // zero-minute stay is free
        int days = minutes / 1440, rest = minutes % 1440;
        double restFee = Math.min(((rest + 59) / 60) * ratePerHour(), dailyMax());
        return days * dailyMax() + restFee;
    }
    @Override public boolean equals(Object o) { return o instanceof Vehicle v && regNo.equals(v.regNo); }
    @Override public int hashCode() { return regNo.hashCode(); }
    @Override public String toString() { return getClass().getSimpleName() + "[" + regNo + "]"; }
}
class TwoWheeler extends Vehicle { TwoWheeler(String r) { super(r); } protected double ratePerHour() { return 10; } }
```

```
class Car extends Vehicle { Car(String r) { super(r); } protected double ratePerHour() { return 30; } }
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String r) { super(r); }
    public double chargingFee(int kWh) { return kWh * 12; }
}

enum UserType { STUDENT, STAFF, VISITOR }

class ParkingTicket {
    private static int counter = 1000;                  // static: shared by every ticket
    private final int ticketNo;
    private final Vehicle vehicle;
    private final UserType user;
    private final boolean validPass;
    private final int floor, slot;
    private final int entryMinute;                      // minutes since midnight
    ParkingTicket(Vehicle v, UserType u, boolean pass, int floor, int slot, int entryMinute) {
        this.ticketNo = ++counter; this.vehicle = v; this.user = u;
        this.validPass = pass; this.floor = floor; this.slot = slot; this.entryMinute = entryMinute;
    }
    int getTicketNo() { return ticketNo; }
    Vehicle getVehicle() { return vehicle; }
    UserType getUser() { return user; }
    boolean hasFreePass() { return validPass && user != UserType.VISITOR; }
    int getFloor() { return floor; }
    int getSlot() { return slot; }
    int getEntryMinute() { return entryMinute; }
}

class ParkingLot {
    private final Vehicle[][] slots;                    // jagged array
    private final Map<String, ParkingTicket> active = new HashMap<>();
    private double collection = 0;
    ParkingLot(int... slotsPerFloor) {
        slots = new Vehicle[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) slots[f] = new Vehicle[slotsPerFloor[f]];
```

```
    }
    ParkingTicket enter(Vehicle v, UserType u, boolean pass, int entryMinute) {
        if (active.containsKey(v.getRegNo()))
            throw new IllegalStateException("Duplicate registration: " + v.getRegNo());
        for (int f = 0; f < slots.length; f++)
            for (int s = 0; s < slots[f].length; s++)
                if (slots[f][s] == null) {
                    slots[f][s] = v;
                    ParkingTicket t = new ParkingTicket(v, u, pass, f, s, entryMinute);
                    active.put(v.getRegNo(), t);
                    return t;
                }
        throw new IllegalStateException("Parking full");
    }
    double exit(String regNo, int exitMinute, int kWh) {
        ParkingTicket t = active.remove(regNo.toUpperCase());
        if (t == null) throw new NoSuchElementException("No active ticket for " + regNo);
        if (t.getFloor() < 0 || t.getFloor() >= slots.length) throw new IllegalStateException("Invalid slot");
        slots[t.getFloor()][t.getSlot()] = null;
        double fee = t.hasFreePass() ? 0 : t.getVehicle().calculateFee(exitMinute - t.getEntryMinute());
        if (t.getVehicle() instanceof Chargeable c && kWh > 0) fee += c.chargingFee(kWh);
        collection += fee;
        return fee;
    }
    int freeSlots() { int n = 0; for (Vehicle[] fl : slots) for (Vehicle v : fl) if (v == null) n++; return n; }
    double getCollection() { return collection; }
}

public class ParkingSystem {
    static int pass = 0, fail = 0;
    static void check(String id, String desc, Object expected, Object actual) {
        boolean ok = expected.toString().equals(actual.toString());
        if (ok) pass++; else fail++;
        System.out.printf("%-4s %-52s exp=%-18s act=%-18s %s%n", id, desc, expected, actual, ok ?
"PASS" : "FAIL");
    }
```

```
    static String attempt(Runnable r) {
        try { r.run(); return "no exception"; }
        catch (RuntimeException e) { return e.getClass().getSimpleName(); }
    }
    public static void main(String[] args) {
        System.out.println("---- Section A: fee unit tests ----");
        check("T1", "TwoWheeler 130 min (3 hrs x 10)", 30.0, new
TwoWheeler("TN01A1").calculateFee(130));
        check("T2", "Car 60 min (exactly 1 hr, slab boundary)", 30.0, new
Car("TN02B2").calculateFee(60));
        check("T3", "Car 61 min (just over boundary -> 2 hrs)", 60.0, new
Car("TN02B2").calculateFee(61));
        check("T4", "Car 0 min (zero-minute stay)", 0.0, new Car("TN02B2").calculateFee(0));
        check("T5", "Car 600 min (10 hrs=300 capped at 240)", 240.0, new
Car("TN02B2").calculateFee(600));
        check("T6", "Car 1500 min (1 day + 60 min = 240 + 30)", 270.0, new
Car("TN02B2").calculateFee(1500));
        check("T7", "Negative duration", "IllegalArgumentException", attempt(() -> new
Car("X1").calculateFee(-5)));
        check("T8", "Blank registration number", "IllegalArgumentException", attempt(() -> new Car("
")));
        check("T9", "equals: same regNo, different subclass", true, new Car("tn02b2").equals(new
ElectricCar("TN02B2")));

        System.out.println("\n---- Section B: facility behaviour (2 floors: 2 + 1 slots) ----");
        ParkingLot small = new ParkingLot(2, 1);
        ParkingTicket a = small.enter(new Car("TN10A"), UserType.VISITOR, false, 0);
        ParkingTicket b = small.enter(new Car("TN11B"), UserType.VISITOR, false, 0);
        check("T10", "First two slots allocated (floor,slot)", "0,0 0,1", a.getFloor() + "," + a.getSlot() + "
" + b.getFloor() + "," + b.getSlot());
        check("T11", "Ticket numbers unique and increasing", true, b.getTicketNo() == a.getTicketNo()
+ 1);
        ParkingTicket c = small.enter(new Car("TN12C"), UserType.VISITOR, false, 0);
        check("T12", "Last free slot is on floor 1", "1,0", c.getFloor() + "," + c.getSlot());
        check("T13", "Facility full -> exception", "IllegalStateException", attempt(() -> small.enter(new
Car("TN13D"), UserType.VISITOR, false, 0)));
```

```
        check("T14", "Duplicate registration rejected", "IllegalStateException", attempt(() -> small.enter(new Car("tn10a"), UserType.VISITOR, false, 0)));
        check("T15", "Free slots when full", 0, small.freeSlots());
        small.exit("TN11B", 60, 0);
        check("T16", "Slot freed after exit", 1, small.freeSlots());
        check("T17", "Exit of unknown vehicle", "NoSuchElementException", attempt(() -> small.exit("TN99Z", 60, 0)));
        ParkingTicket d = small.enter(new Car("TN14E"), UserType.VISITOR, false, 70);
        check("T18", "Freed slot is reused", "0,1", d.getFloor() + "," + d.getSlot());

        System.out.println("\n---- Section C: day scenario (hand-calculated Rs 240) ----");
        ParkingLot lot = new ParkingLot(3, 3);
        lot.enter(new TwoWheeler("TN01A1"), UserType.VISITOR, false, 540);
        lot.enter(new Car("TN02B2"), UserType.VISITOR, false, 540);
        lot.enter(new Car("TN03C3"), UserType.STUDENT, true, 540);
        lot.enter(new ElectricCar("TN04D4"), UserType.VISITOR, false, 540);
        lot.enter(new Car("TN05E5"), UserType.STAFF, false, 540);
        double f1 = lot.exit("TN01A1", 540 + 130, 0);
        double f2 = lot.exit("TN02B2", 540 + 60, 0);
        double f3 = lot.exit("TN03C3", 540 + 300, 0);
        double f4 = lot.exit("TN04D4", 540 + 61, 10);
        double f5 = lot.exit("TN05E5", 540, 0);
        System.out.println("Two-wheeler 130 min : Rs " + f1 + "  (hand: 3 x 10 = 30)");
        System.out.println("Car 60 min          : Rs " + f2 + "  (hand: 1 x 30 = 30)");
        System.out.println("Student pass 300 min: Rs " + f3 + "   (hand: free)");
        System.out.println("EV 61 min + 10 kWh  : Rs " + f4 + " (hand: 2 x 30 + 10 x 12 = 180)");
        System.out.println("Staff, 0 min        : Rs " + f5 + "   (hand: 0)");
        check("T19", "Total collection for the day", 240.0, lot.getCollection());
        System.out.println("\nSummary: " + pass + " passed, " + fail + " failed");
    }
}
```

11.2 Execution output and test table

Output – 19 of 19 tests passed

```
---- Section A: fee unit tests ----
```

```
T1   TwoWheeler 130 min (3 hrs x 10)                exp=30.0          act=30.0          PASS
T2   Car 60 min (exactly 1 hr, slab boundary)       exp=30.0          act=30.0          PASS
T3   Car 61 min (just over boundary -> 2 hrs)       exp=60.0          act=60.0          PASS
T4   Car 0 min (zero-minute stay)                   exp=0.0           act=0.0           PASS
T5   Car 600 min (10 hrs=300 capped at 240)         exp=240.0         act=240.0         PASS
T6   Car 1500 min (1 day + 60 min = 240 + 30)       exp=270.0         act=270.0         PASS
T7     Negative  duration                                             exp=IllegalArgumentException
act=IllegalArgumentException PASS
T8     Blank  registration  number                                     exp=IllegalArgumentException
act=IllegalArgumentException PASS
T9   equals: same regNo, different subclass         exp=true          act=true          PASS

---- Section B: facility behaviour (2 floors: 2 + 1 slots) ----
T10  First two slots allocated (floor,slot)         exp=0,0 0,1       act=0,0 0,1       PASS
T11  Ticket numbers unique and increasing           exp=true          act=true          PASS
T12  Last free slot is on floor 1                   exp=1,0           act=1,0           PASS
T13  Facility full -> exception                     exp=IllegalStateException act=IllegalStateException
PASS
T14  Duplicate registration rejected                exp=IllegalStateException act=IllegalStateException
PASS
T15  Free slots when full                           exp=0             act=0             PASS
T16  Slot freed after exit                          exp=1             act=1             PASS
T17   Exit  of  unknown  vehicle                                      exp=NoSuchElementException
act=NoSuchElementException PASS
T18  Freed slot is reused                           exp=0,1           act=0,1           PASS

---- Section C: day scenario (hand-calculated Rs 240) ----
Two-wheeler 130 min : Rs 30.0  (hand: 3 x 10 = 30)
Car 60 min          : Rs 30.0  (hand: 1 x 30 = 30)
Student pass 300 min: Rs 0.0   (hand: free)
EV 61 min + 10 kWh  : Rs 180.0 (hand: 2 x 30 + 10 x 12 = 180)
Staff, 0 min        : Rs 0.0   (hand: 0)
T19  Total collection for the day                   exp=240.0         act=240.0         PASS

Summary: 19 passed, 0 failed
```

Group	Test	Expected	Actual	Result
Normal	Two-wheeler 130 min	Rs 30	Rs 30	Pass
Normal	Allocation order of first two cars	0,0 then 0,1	0,0 then 0,1	Pass
Boundary	Car 60 min (exact slab)	Rs 30	Rs 30	Pass
Boundary	Car 61 min (just over)	Rs 60	Rs 60	Pass
Boundary	Zero-minute stay	Rs 0	Rs 0	Pass
Boundary	Last free slot (floor 1)	1,0	1,0	Pass
Boundary	Daily maximum (600 min)	Rs 240	Rs 240	Pass
Invalid	Facility full	IllegalStateException	IllegalStateException	Pass
Invalid	Duplicate registration	IllegalStateException	IllegalStateException	Pass
Invalid	Exit of unknown vehicle	NoSuchElementException	NoSuchElementException	Pass
Invalid	Negative duration / blank regNo	IllegalArgumentException	IllegalArgumentException	Pass

11.3 Hand calculation for a day's collection

Vehicle	Stay / extras	Working	Fee (Rs)
TN01A1 two-wheeler (visitor)	130 min	$\lceil 130/60 \rceil = 3\ \text{h} \times 10$	30
TN02B2 car (visitor)	60 min	1 h × 30	30
TN03C3 car (student, valid pass)	300 min	free	0

Vehicle	Stay / extras	Working	Fee (Rs)
TN04D4 electric car (visitor)	61 min, 10 kWh	2 h × 30 + 10 × 12 = 60 + 120	180
TN05E5 car (staff)	0 min	zero-minute stay	0
Total		30 + 30 + 0 + 180 + 0	**240**

The program printed a day total of Rs 240.0, equal to the hand calculation (test T19).

11.4 Evaluation of the design

- **Encapsulation:** slots, active tickets and regNo are private; all changes go through methods that validate first.
- **Inheritance:** TwoWheeler, Car and ElectricCar reuse calculateFee() and differ only in ratePerHour() (plus charging for EVs).
- **Polymorphism:** exit() calls getVehicle().calculateFee(...) without any type test.
- **Abstraction:** Vehicle (abstract) and Chargeable (interface) separate what a vehicle must do from how.
- **Limitation:** a single time model (minutes since midnight) cannot express fee rules that depend on the clock (peak hours) or on dates. Task 12 had to add an overload to Vehicle for this. Also, fees use double; real billing should use BigDecimal or integer paise.

11.5 Alternative concept – ArrayList or HashMap for active tickets

With an array or ArrayList, exit processing must search for the ticket by registration number – O(n). With a HashMap<String, ParkingTicket> keyed by registration number it is O(1) on average. The benchmark below (2,000 random look-ups, Java 21) measures this on the same data.

Bench.java

```
import java.util.*;
public class Bench {
   static long sink;
   public static void main(String[] a){
      int[] sizes = {1_000, 10_000, 50_000};
      System.out.printf("%-8s %-16s %-16s %-16s%n","n","Array scan (ms)","ArrayList (ms)","HashMap (ms)");
      for (int n : sizes) {
         String[] arr = new String[n]; ArrayList<String> list = new ArrayList<>(); HashMap<String,Integer> map = new HashMap<>();
```

```
            for (int i = 0; i < n; i++){ String r = "TN" + i; arr[i]=r; list.add(r); map.put(r,i); }
            String[] keys = new String[2000]; Random rnd = new Random(42);
            for (int i = 0; i < keys.length; i++) keys[i] = "TN" + rnd.nextInt(n);
            for (int w=0; w<2; w++) run(arr,list,map,keys,false);   // warm-up
            long[] t = run(arr,list,map,keys,true);
            System.out.printf("%-8d %-16.2f %-16.2f %-16.2f%n", n, t[0]/1e6, t[1]/1e6, t[2]/1e6);
        }
    }
    static long[] run(String[] arr, ArrayList<String> list, HashMap<String,Integer> map, String[] keys,
boolean m){
        long t0=System.nanoTime();
        for (String k: keys){ for (int i=0;i<arr.length;i++) if (arr[i].equals(k)){ sink+=i; break; } }
        long t1=System.nanoTime();
        for (String k: keys) sink += list.indexOf(k);
        long t2=System.nanoTime();
        for (String k: keys) sink += map.get(k);
        long t3=System.nanoTime();
        return new long[]{t1-t0,t2-t1,t3-t2};
    }
}
```

Measured output (milliseconds for 2,000 look-ups; values vary by machine)

```
n       Array scan (ms)  ArrayList (ms)   HashMap (ms)
1000     13.96            2.06             0.07
10000    25.87            24.58            0.07
50000    174.12           162.13           0.11
```

Structure	Find ticket on exit	Space for n vehicles	Remarks
Array of tickets	O(n) – 174 ms at n = 50,000	O(n), compact	Fixed capacity; simple
ArrayList	O(n) – 162 ms at n = 50,000	O(n) plus spare capacity	Grows; deletion shifts elements O(n)

Structure	Find ticket on exit	Space for n vehicles	Remarks
HashMap (used)	O(1) average – 0.11 ms at n = 50,000	O(n) with larger constant (entry objects, table)	Hash-based; needs correct equals()/hashCode()

The measurements agree with the theory: the linear structures grow roughly in proportion to n (13.96 ms → 174 ms is about 12× for 50× more data, partly hidden by JIT warm-up), while the HashMap time is essentially flat. The HashMap spends more memory per entry, which is acceptable for a facility of at most 500 slots.

Task 12 – Re-evaluate and Adapt the Design

Bloom's level: K5 – Evaluate | Marks: 8

A. Impact analysis (2 marks)

What changed: two new vehicle categories (e-bike with a flat fee; delivery van charged double the car rate between 9 AM and 5 PM), a new one-day visitor pass, and an expansion from 3 to 5 floors with a different slot count on each floor.

Component	Affected?	Reason
Vehicle hierarchy	Extended	EBike and DeliveryVan are new subclasses.
Vehicle.calculateFee(int)	Not enough	Van pricing needs the entry clock time, not only the duration.
Pass handling	New	A one-day pass is a new kind of pass with a date rule; a monthly pass has a month rule.
Floor array	Must be jagged	A rectangular array wastes slots when floors differ; a 1-D index formula (f × S + s) breaks.
ParkingLot allocation loops	Unchanged	The loops already use slots[f].length, so any floor size works.

Classes that must be **modified** rather than extended: only Vehicle, and only by adding one method (a time-aware overload). All other changes are new classes.

B. Evaluation of the original design (2 marks)

- **New vehicle types by new subclasses only?** Yes for the e-bike (it overrides calculateFee(int)). For the van the base signature lacked the clock time, so one additive method had to be added to Vehicle.
- **Pricing rules in one place or spread out?** They are spread across subclasses (one rule per class) with the common slab logic in Vehicle. This is cohesive per type, but a rule that changes for all types (for example the daily cap) is one edit, while a rule that differs by type is one class – acceptable for three or four types. With many rules, a separate PricingRule abstraction would centralise them.
- **Existing tested classes unchanged?** TwoWheeler, Car, ElectricCar and ParkingLot are untouched; regression tests R1–R3 below confirm identical output.

C. Decision: retain and extend (modify minimally) (2 marks)

Decision: RETAIN the design with one additive modification. The original abstraction (abstract Vehicle, polymorphic fee, jagged slot array) already supports the new requirements. Redesigning with a PricingRule hierarchy would also work, but it would rewrite all the classes that already work and are tested, for no functional gain now. Changes made:

1. Vehicle gets one new method, calculateFee(LocalTime entry, int minutes), whose default implementation simply calls the old calculateFee(int). Every existing subclass inherits identical behaviour.
2. EBike extends Vehicle and overrides calculateFee(int) to return a flat Rs 20 per visit (0 for a zero-minute stay).
3. DeliveryVan extends Vehicle and overrides calculateFee(LocalTime, int): for each started hour, the rate is doubled if that hour starts between 09:00 and 17:00. The rule is applied from the start time of each billed hour (an assumption stated in the report).
4. Pass is a new interface with MonthlyPass and OneDayPass implementations; the facility asks pass.isValidOn(date) and charges nothing when it is valid.
5. Facility uses a jagged array with 5 floors of 20, 30, 40, 50 and 60 slots (capacity 200).

Task12Demo.java

```
import java.time.*;
import java.util.*;
interface Chargeable { double chargingFee(int kWh); }
abstract class Vehicle {
    private final String regNo;
```

```
    protected Vehicle(String regNo) { this.regNo = regNo.trim().toUpperCase(); }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();
    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Duration cannot be negative");
        return ((minutes + 59) / 60) * ratePerHour();
    }
    public double calculateFee(LocalTime entry, int minutes) { return calculateFee(minutes); }
}
class TwoWheeler extends Vehicle { TwoWheeler(String r){super(r);} protected double ratePerHour(){return 10;} }
class Car extends Vehicle { Car(String r){super(r);} protected double ratePerHour(){return 30;} }
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String r){super(r);} public double chargingFee(int kWh){return kWh*12;}
}
class EBike extends Vehicle {
    static final double FLAT_FEE = 20;
    EBike(String r){super(r);}
    @Override protected double ratePerHour(){ return FLAT_FEE; }
    @Override public double calculateFee(int minutes){
        if (minutes < 0) throw new IllegalArgumentException("Duration cannot be negative");
        return minutes == 0 ? 0 : FLAT_FEE;              // flat fee per visit
    }
}
class DeliveryVan extends Vehicle {
    private static final int PEAK_START = 9 * 60, PEAK_END = 17 * 60;
    DeliveryVan(String r){super(r);}
    @Override protected double ratePerHour(){ return 30; }    // off-peak = car rate
    @Override public double calculateFee(LocalTime entry, int minutes){
        if (minutes < 0) throw new IllegalArgumentException("Duration cannot be negative");
        double fee = 0;
        int start = entry.getHour() * 60 + entry.getMinute();
        for (int done = 0; done < minutes; done += 60) {   // every started hour
            int t = (start + done) % 1440;                 // rate fixed by the hour's start time
            boolean peak = t >= PEAK_START && t < PEAK_END;
            fee += peak ? 2 * ratePerHour() : ratePerHour();
```

```
        }
        return fee;
    }
}
interface Pass { boolean isValidOn(LocalDate day); String describe(); }
class MonthlyPass implements Pass {
    private final YearMonth month;
    MonthlyPass(YearMonth m){ month = m; }
    public boolean isValidOn(LocalDate d){ return YearMonth.from(d).equals(month); }
    public String describe(){ return "MonthlyPass(" + month + ")"; }
}
class OneDayPass implements Pass {
    private final LocalDate day;
    OneDayPass(LocalDate d){ day = d; }
    public boolean isValidOn(LocalDate d){ return day.equals(d); }
    public String describe(){ return "OneDayPass(" + day + ")"; }
}
 class Facility {
    private final Vehicle[][] slots;
    private final Map<String,Object[]> active = new HashMap<>();   // reg -> {vehicle, entry, pass, f, s}
    Facility(int... perFloor){
        slots = new Vehicle[perFloor.length][];
        for (int f = 0; f < perFloor.length; f++) slots[f] = new Vehicle[perFloor[f]];
    }
    int capacity(){ int n=0; for (Vehicle[] f: slots) n += f.length; return n; }
    String enter(Vehicle v, LocalDateTime in, Pass pass){
        for (int f = 0; f < slots.length; f++)
            for (int s = 0; s < slots[f].length; s++)
                if (slots[f][s] == null){ slots[f][s] = v; active.put(v.getRegNo(), new Object[]{v,in,pass,f,s});
return "F"+f+"-S"+s; }
        throw new IllegalStateException("Parking full");
    }
    double exit(String reg, LocalDateTime out){
        Object[] r = active.remove(reg);
        Vehicle v = (Vehicle) r[0]; LocalDateTime in = (LocalDateTime) r[1]; Pass p = (Pass) r[2];
        slots[(int) r[3]][(int) r[4]] = null;
```

```
        if (p != null && p.isValidOn(in.toLocalDate())) return 0;
        int minutes = (int) Duration.between(in, out).toMinutes();
        return v.calculateFee(in.toLocalTime(), minutes);     // dynamic dispatch picks the right rule
    }
}
public class Task12Demo {
    static int pass = 0, fail = 0;
    static void check(String id, String d, double exp, double act){
        boolean ok = exp == act; if (ok) pass++; else fail++;
        System.out.printf("%-4s %-58s exp=%-7.1f act=%-7.1f %s%n", id, d, exp, act, ok ? "PASS":"FAIL");
    }
    public static void main(String[] args){
        LocalDate day = LocalDate.of(2026, 11, 10);
        Facility fac = new Facility(20, 30, 40, 50, 60);        // 5 floors, different sizes
        System.out.println("Facility capacity (20+30+40+50+60) = " + fac.capacity());
        System.out.println("---- Regression: original classes still behave identically ----");
        check("R1","TwoWheeler 130 min", 30, new TwoWheeler("A").calculateFee(130));
        check("R2","Car 130 min", 90, new Car("B").calculateFee(130));
        check("R3","Car 130 min via NEW overload (default path)", 90, new Car("B").calculateFee(LocalTime.of(10,0),130));
        System.out.println("---- New categories ----");
        check("N1","E-bike 15 min (flat fee)", 20, new EBike("EB1").calculateFee(15));
        check("N2","E-bike 5 hours (still flat)", 20, new EBike("EB1").calculateFee(300));
        check("N3","Van 10:00-12:00 peak (2 x 60)", 120, new DeliveryVan("V1").calculateFee(LocalTime.of(10,0),120));
        check("N4","Van 18:00-19:00 off-peak (1 x 30)", 30, new DeliveryVan("V1").calculateFee(LocalTime.of(18,0),60));
        check("N5","Van 08:30-11:30 (30 + 60 + 60)", 150, new DeliveryVan("V1").calculateFee(LocalTime.of(8,30),180));
        check("N6","Van 16:30-17:30 (hour starts 16:30 -> peak 60)", 60, new DeliveryVan("V1").calculateFee(LocalTime.of(16,30),60));
        System.out.println("---- Facility + passes ----");
        LocalDateTime in = day.atTime(10,0);
        String s1 = fac.enter(new Car("CAR1"), in, null);
        String s2 = fac.enter(new Car("CAR2"), in, new OneDayPass(day));
```

```
        String s3 = fac.enter(new DeliveryVan("VAN1"), in, null);
        String s4 = fac.enter(new EBike("EB9"), in, null);
        System.out.println("Allocated slots: CAR1=" + s1 + " CAR2=" + s2 + " VAN1=" + s3 + " EB9="
+ s4);
        check("F1","Visitor car, no pass, 2 h", 60, fac.exit("CAR1", in.plusHours(2)));
        check("F2","Visitor car with valid one-day pass, 5 h", 0, fac.exit("CAR2", in.plusHours(5)));
        check("F3","Van 10:00 for 2 h through facility", 120, fac.exit("VAN1", in.plusHours(2)));
        check("F4","E-bike 3 h through facility", 20, fac.exit("EB9", in.plusHours(3)));
        Pass old = new OneDayPass(day.minusDays(1));
        fac.enter(new Car("CAR3"), in, old);
        check("F5","Expired one-day pass (yesterday) -> full fee, 1 h", 30, fac.exit("CAR3",
in.plusHours(1)));
        System.out.println("\nSummary: " + pass + " passed, " + fail + " failed");
    }
}
```

Output – 14 of 14 tests passed

```
Facility capacity (20+30+40+50+60) = 200
---- Regression: original classes still behave identically ----
R1   TwoWheeler 130 min                                exp=30.0    act=30.0    PASS
R2   Car 130 min                                   exp=90.0    act=90.0    PASS
R3   Car 130 min via NEW overload (default path)          exp=90.0    act=90.0    PASS
---- New categories ----
N1   E-bike 15 min (flat fee)                      exp=20.0    act=20.0    PASS
N2   E-bike 5 hours (still flat)                   exp=20.0    act=20.0    PASS
N3   Van 10:00-12:00 peak (2 x 60)                    exp=120.0   act=120.0   PASS
N4   Van 18:00-19:00 off-peak (1 x 30)                exp=30.0    act=30.0    PASS
N5   Van 08:30-11:30 (30 + 60 + 60)                   exp=150.0   act=150.0   PASS
N6   Van 16:30-17:30 (hour starts 16:30 -> peak 60)        exp=60.0    act=60.0    PASS
---- Facility + passes ----
Allocated slots: CAR1=F0-S0 CAR2=F0-S1 VAN1=F0-S2 EB9=F0-S3
F1   Visitor car, no pass, 2 h                     exp=60.0    act=60.0    PASS
F2   Visitor car with valid one-day pass, 5 h           exp=0.0     act=0.0     PASS
F3   Van 10:00 for 2 h through facility                exp=120.0   act=120.0   PASS
F4   E-bike 3 h through facility                   exp=20.0    act=20.0    PASS
```

```
F5  Expired one-day pass (yesterday) -> full fee, 1 h        exp=30.0   act=30.0   PASS

Summary: 14 passed, 0 failed
```

D. Evidence-based defence (2 marks)

Criterion	Evidence
Existing classes changed	1 of 6 (Vehicle: +1 method, no existing line edited). TwoWheeler, Car, ElectricCar, Chargeable and the facility loops: 0 changes.
New classes added	EBike, DeliveryVan, Pass, MonthlyPass, OneDayPass, Facility (jagged 5-floor version): 6.
Ease of adding another type	One new subclass, overriding only the method whose rule differs; no edits to callers (dynamic dispatch).
Test results	Regression R1–R3 identical to before; new-category tests N1–N6 and facility/pass tests F1–F5 all pass (14 of 14).
Trade-offs	Pricing rules remain spread across classes; Vehicle now knows about LocalTime; a PricingRule strategy would centralise rules at the cost of a larger refactor. If five or more distinct rules appear, that refactor becomes worth doing.

The decision follows from the evidence: since a one-method additive change plus new subclasses satisfied every changed requirement and left every earlier test green, retaining and extending the design was correct. A full redesign would have cost more and risked regressions without making any new requirement easier.

Conclusion

The Campus Smart-Parking System was analysed, designed and implemented using encapsulation, inheritance, polymorphism and abstraction. All programs were compiled and executed; the supplied Complete-the-Code outputs match the expected output exactly, 19 tests passed in the main system, and 14 passed after the Task 12 change. The design was retained and extended, with one additive method in one existing class, because the evidence showed that the original abstractions already accommodated the new requirements.